University of Tunis El Manar
National Engineering School of Tunis
Department of Information and
Communication Technologies

End-of-year project

From Network Diagrams to Deployable Cloud Infrastructure: A Multimodal AI-Based Approach

Elaborated by:

Dhia Eddine BARHOUMI

Wael HAMMALI

Supervised by:

Ms Wafa MEFTEH

Class: 2nd year software student

Year: 2025/2026

Acknowledgment

First and foremost, I would like to express my deepest gratitude to the National Engineering School of Tunis, ENIT, for the quality of its academic training, the richness of its engineering environment, and the valuable learning opportunities it has provided throughout my studies. The knowledge, discipline, and technical skills acquired at ENIT have been essential in the realization of this end-of-year project.

I would also like to sincerely thank the Department of Information and Communication Technologies for the continuous support, guidance, and academic framework offered during our training. This project is the result of the solid foundation built through the courses, projects, and practical experiences provided by the department.

My sincere thanks go to Ms. Wafa Mefteh for her supervision, availability, and valuable advice throughout this work.

Abstract

This report presents Net2Terraform WebInterface, an intelligent platform designed to transform network topology intent into deployable AWS Infrastructure as Code. The system supports two complementary input modes: image-based topology analysis and conversational architecture description.

The image pipeline combines object detection, topology link extraction, and OCR-based label interpretation to produce structured network information. The conversational pipeline relies on architecture extraction, interactive validation, and context-grounded generation to improve consistency and reduce ambiguity. Both pipelines converge to Terraform generation, with optional deployment orchestration and monitoring.

The implementation follows a modular backend architecture with clear service decomposition, provider-aware language model routing, and retrieval-assisted decision support. To improve detection performance, a dedicated dataset engineering process was conducted using synthetic data generation and Roboflow-assisted annotation workflows. The final dataset includes 1200 images, including 1000 generated samples.

Evaluation results show strong detection performance, low class confusion, good generation reliability, and practical end-to-end usability. The project demonstrates that combining computer vision, grounded conversational intelligence, and cloud automation can significantly reduce the gap between topology design and real infrastructure deployment.
Keywords: Infrastructure as Code, Terraform, AWS, YOLO, OCR, Retrieval-Augmented Generation, FastAPI, Network Topology Automation.

Acronyms

API	Application Programming Interface
AWS	Amazon Web Services
BM25	Best Matching 25
CIDR	Classless Inter-Domain Routing
CPU	Central Processing Unit
EC2	Elastic Compute Cloud
FAISS	Facebook AI Similarity Search
GPU	Graphics Processing Unit
IaC	Infrastructure as Code
IGW	Internet Gateway
JSON	JavaScript Object Notation
LLM	Large Language Model
NAT	Network Address Translation
OBB	Oriented Bounding Box
OCR	Optical Character Recognition
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SG	Security Group
SSH	Secure Shell
TGW	Transit Gateway
UI	User Interface
VPC	Virtual Private Cloud
YOLO	You Only Look Once

Contents

List of Figures

List of Tables

General Introduction

We are living through a fundamental shift in how digital infrastructure is built and managed. Where network systems once meant racks of physical hardware painstakingly configured by hand, today's environments run on virtualized resources that can be created or removed in minutes. Cloud platforms have made elastic provisioning the new normal, and Infrastructure as Code has taken things a step further by turning infrastructure itself into something that can be written, versioned, and deployed like software. The result is a new generation of systems that are faster to build, easier to reproduce, and accessible to a much broader audience than ever before.

Yet despite this progress, the way people learn network design has remained largely unchanged. Aspiring engineers still sketch routers, switches, servers, and client devices as visual diagrams, often relying on simulation tools to understand how everything connects. This is a reasonable starting point, since visualizing a topology helps build intuition in a way that documentation alone cannot. The problem is that a clear diagram and a working deployment are not the same thing. A topology that appears complete on paper may still miss critical details such as IP addressing, network segmentation, security boundaries, and cloud-specific constraints.

This gap is precisely what the present project aims to address. Its objective is to create a platform capable of taking a network topology, regardless of how it is expressed, and transforming it into real and deployable cloud infrastructure. The platform supports two modes of interaction. Users may provide a topology image, which is analyzed through computer vision and OCR in order to identify components and extract their labels. Alternatively, users may describe the topology in plain text, allowing the system to build the components, links, and deployment intent through a structured input process. In both cases, the collected information is normalized into a shared internal model before being translated into concrete AWS infrastructure artifacts.

What distinguishes this approach is that topology understanding and infrastructure generation are not treated as separate concerns. Instead of passing information from one independent tool to another, the platform integrates interpretation, validation, normalization, and cloud translation into a single coherent workflow. The main output is Terraform, which is widely used for cloud provisioning, while Ansible is included as an optional layer for host-level configuration after deployment.

The remainder of this report is organized as follows. Chapter 1 introduces the technical and scientific background, including cloud computing, Infrastructure as Code, large language models, Retrieval-Augmented Generation, object detection, and OCR.

Chapter 2 is dedicated to the technical realization of the Solution. Chapter 3 discusses testing, results, and their interpretation.

Chapter 1

State of the Art and Technological Foundations

1.1 Introduction

Before building our solution, it was important to understand the main technologies and ideas behind it. This chapter presents a simple overview of the key concepts used in this project, from cloud computing and Infrastructure as Code to recent advances in artificial intelligence.

Rather than going too deep into theory, the goal here is to highlight what is really useful for our problem and to explain why certain choices were made. We also take a look at existing tools and approaches to better position our work and understand what is already possible, and what still needs improvement.

1.2 Large Language Models and Retrieval-Augmented Generation

This section discusses the language processing techniques that support the proposed platform, especially during topology interpretation and Terraform generation. It explains how combining language models with retrieval helps reduce unsupported generation and keeps the output closer to the project requirements.

1.2.1 Large Language Models

The Transformer architecture is the basis of modern language models and has revolutionized machine learning techniques for dealing with language data due to its use of attention mechanisms. Attention is very different from previous approaches, such as word embeddings, because it allows a model to understand the connections between words in a sequence, regardless of their positions. As a result, attention has brought significant performance and efficiency improvements to language modeling techniques. A language model is based on an encoder-decoder architecture. Encoder-based models are used to analyze texts and are therefore useful for tasks such as classification and semantic analysis. On the other

hand, decoder-based models can generate text because they predict tokens for the next position. Language models are pre-trained on vast amounts of textual data and then fine-tuned for a particular application area.[1].

1.2.2 Limitations of Standard LLMs

However, despite their capability of generating responses, there are some limitations inherent to typical LLMs. The first one is that these systems tend to generate fluent answers to questions that may be inaccurate. Indeed, these answers are generated without referencing any facts or reliable sources since the answer is built by analyzing statistical patterns.

Another important limitation is the limited size of context window for LLMs. In other words, there is an absolute limit on the volume of textual input processed to create an answer. Even in case when the size of the context window is increased, it is not possible to find the sources used for the generation of an answer.

For these reasons, using a standard LLM alone is often insufficient in technical applications. This is the main motivation behind retrieval-based approaches, which extend LLMs by connecting them to external knowledge sources before generating a response[1].

1.2.3 Retrieval-Augmented Generation

RAG refers to a technology that improves the abilities of language models by linking them to an external source of knowledge at the time of inference. Instead of relying solely on the model's knowledge gained during training, the system fetches relevant documents based on the user input and uses them as the context when producing the output. In doing so, RAG delivers responses that are precise and knowledge-based. Knowledge is stored in the form of short text pieces known as chunks, which have been embedded in vectors. Therefore, the system is able to find similarities in the contents by comparing their meaning rather than keywords. The most relevant chunks are then fetched once the query comes, and they are included in the prompt together with the model's knowledge. Thus, the combination of the two approaches helps reduce hallucination issues while improving the factual precision of the generated text.[1].

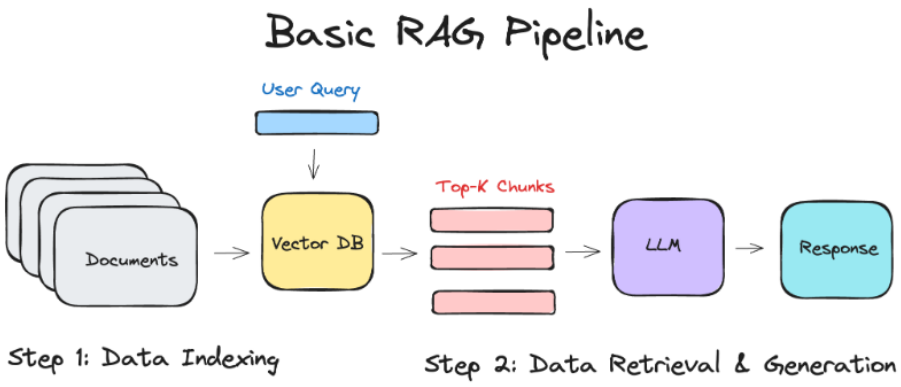

Figure 1.1: Basic Rag Pipeline

1.2.4 Why RAG Was Chosen for This Project

In this project, infrastructure generation depends on domain-specific knowledge that must be explicit and controllable. The system needs to interpret topology inputs, apply mapping rules, and select appropriate AWS design patterns. A standard LLM alone cannot provide the level of reliability and consistency required for this kind of task.

By integrating retrieval, the platfor can draw from a structured knowledge base containing mapping rules, security principles, and cloud design patterns. This keeps the generation process grounded and predictable. RAG is therefore used to support interpretation and planning, while the final infrastructure output remains governed by deterministic translation logic.

1.2.5 Architectural View of Retrieval-Augmented Generation

Retrieval-Augmented Generation systems are best understood as two complementary parts working together: an **offline knowledge preparation pipeline** that runs in the background, and an **online reasoning pipeline** that activates the moment a user submits a query. This distinction matters because the quality of what gets generated depends as much on how knowledge is organized and retrieved as it does on the language model itself.

The **offline phase** is where raw documentation gets shaped into something the system can actually search. It starts with collecting documents, extracting their text, and cleaning it up. The content is then broken into smaller units called **chunks**, a necessary step since dumping an entire document into a retrieval system tends to dilute precision. Smaller segments make it far easier to isolate a specific rule, definition, or design pattern. Each chunk is then passed through an embedding model that converts it into a dense vector, and those vectors are stored in a retrieval index. From that point on, the system can search by semantic meaning rather than just matching keywords.

The **online phase** is the moment a user asks a question, the online phase starts.

Instead of directly sending the query to the language model, it first looks up the index knowledge base for the most relevant chunks. The candidate segments can then undergo a reranking step to weed out anything that is only loosely related leaving behind a tight set of context segments. The model will not only generate from memory but also utilize grounded evidence to respond based on the context embedded into the prompt.

This separation comes with several practical advantages worth noting. The responses generated become much more factual because the model has references to refer to, rather than just what it stored in training. Likewise, as any output claim can be traced back to a specific retrieved chunk, the process is also traceable in principle at least. Most importantly, the system is flexible: to update the knowledge base, you do not need to retrain it, only refresh the index. In technical areas where correctness and consistency are valued over style and elegance, these properties are especially useful.

In practice, the retrieval layer can itself be multi-tiered. The semantic search step deals with conceptual similarity. The second step, ranking or filtering, comes into play if the query uses jargon, abbreviations, or other structure-specific expressions. This is to avoid confusion (from simpler look-up). By the time the query reaches the language processor, it has already been subjected to information-processing through a design.

RAG should not be considered merely as "LLM + documents," but rather as a modular architecture for grounded reasoning. We can think of it as a series of links, where the effectiveness of the model is as good as the weakest link. This can be document preparation, chunking strategy, embedding quality, retrieval, reranking, or prompt construction. With respect to the generation of the technical infrastructure, this viewpoint is particularly relevant: it explains how a system can remain knowledge-aware and operationally reliable at the same time.

1.3 Choice of Object Detection Framework

This section explains the choice of the object detection framework used to analyze network topology images. It focuses on why YOLOv8 was selected and how its architecture supports accurate detection of network components and links.

1.3.1 Introduction to the YOLO Ecosystem

One of the vital aspects of my project is object detection which converts the scanned network diagrams into machines-readable nodes. A very important trait of a system that can automate infrastructure is localization and categorization of features simultaneously (Solomon and Breckon). We selected the You Only Look Once (YOLO) family of architectures due to their industry-leading trade-off between inference speed and mean Average Precision (mAP). This project utilizes YOLOv8 by Ultralytics, and this choice was made

for technical alignment with "NetDevOps" automation[6].

1.3.2 Evolutionary Context and Selection Logic

The YOLO lineage has undergone significant paradigm shifts in neural network design. Rosebrock [5] emphasizes that for real-time applications, the architecture must minimize computational overhead without sacrificing spatial awareness.

- **YOLOv5:** Added PyTorch-native workflow but uses anchor-based detection, which suffers from high variance of hand-drawn stroke widths seen in scanned topologies.
- **YOLOv8 (Selected):** Introduced an **Anchor-Free** paradigm. By predicting the object center directly rather than offsets from pre-defined boxes, it handles the irregular aspect ratios of routers and switches more effectively [6].
- **YOLO11/26 (Emerging):** While newer versions offer NMS-free inference for edge devices, the stability and extensive pre-trained weight libraries of YOLOv8 make it the optimal choice for a backend cloud-logic pipeline.

1.3.3 Architectural Justifications

Detailed Architecture of YOLOv8

The internal structure of YOLOv8 represents a significant leap in feature extraction efficiency. According to the review by Hidayatullah et al. [2] the architecture is divided into three functional segments:

1. **The Backbone (Feature Extraction):** Utilizes a modified CSPDarknet53 to capture hierarchical features.[2]
2. **The Neck (Feature Fusion):** It uses a PANet network that fuses features from different stages of the backbone..[2]
3. **The Head (Prediction):** Implements a **Decoupled Head** for independent processing of classification and regression.[2]

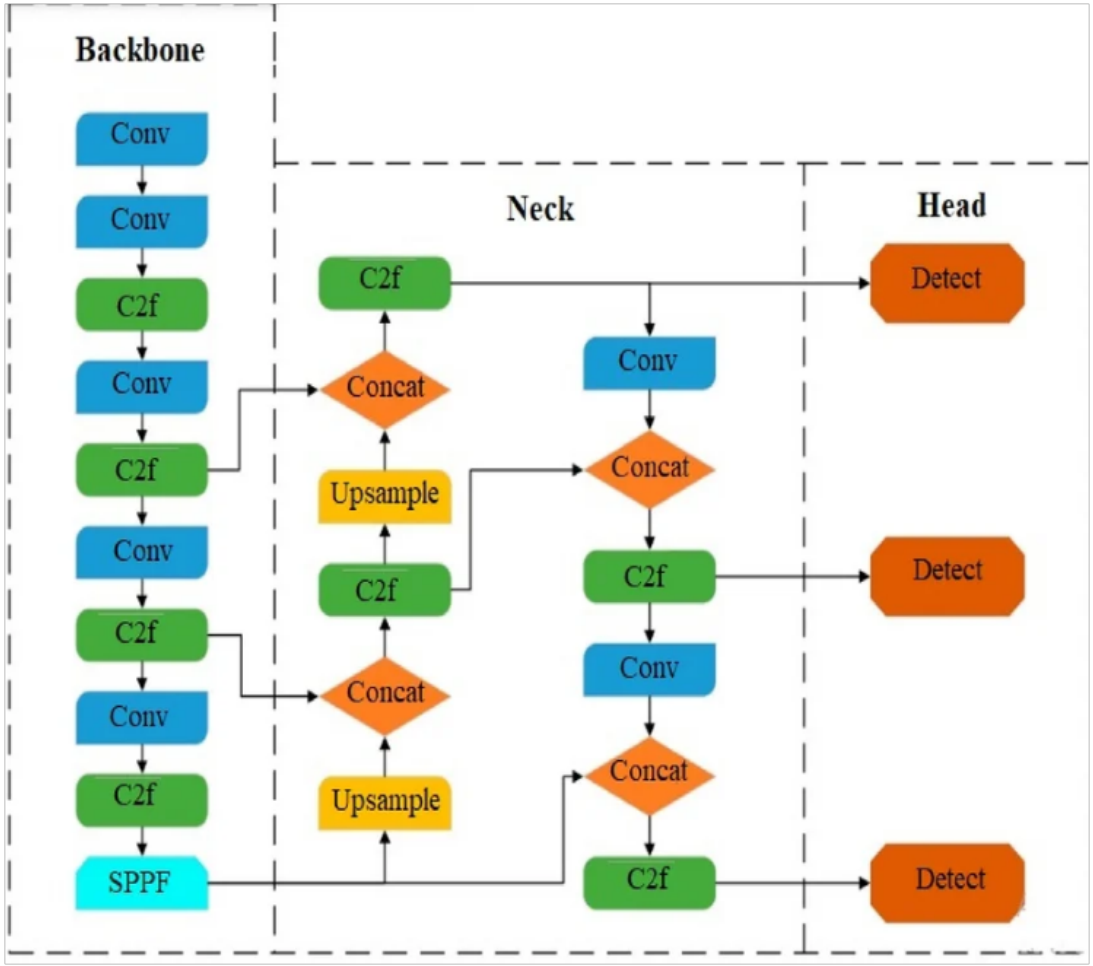

Figure 1.2: The modular architecture of YOLOv8 showing the Backbone, Neck, and Head layers.

Internal Composition of Primary Blocks

Conv Block (CBS) The **CBS** (Convolution, Batch Normalization, SiLU) block is the fundamental unit. YOLOv8 adopts the **SiLU** (Sigmoid Linear Unit) activation function to ensure smooth gradient propagation:

$$f(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \tag{1.1}$$

Unlike standard ReLU, SiLU prevents the "dying neuron" problem, which is critical when detecting fine-grained details in complex network icon sets.

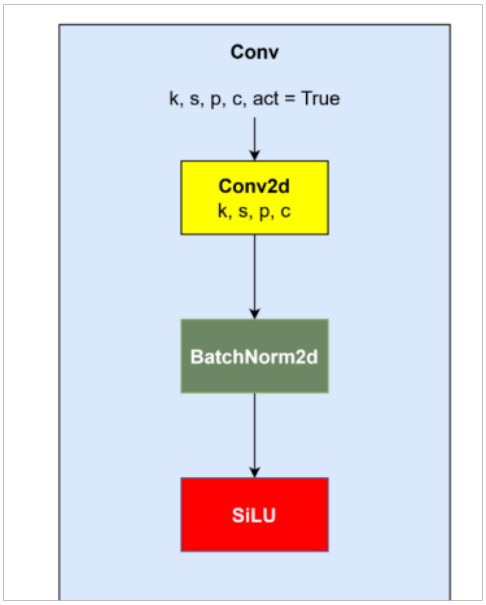

Figure 1.3: Structure interne du bloc CBS (Conv-BN-SiLU) de YOLOv8. Adapté de [2].

Analysis of the C2f Data Flow As illustrated in Figure 1.4, the C2f module implements a gradient flow enhancement strategy. By concatenating the outputs of multiple bottleneck stages, the model maintains "gradient diversity."

For our network topology parsing, this is essential: the initial branches preserve the overall geometric structure of the network lines (links), while the deeper bottleneck branches focus on the specific textures required to classify device types (nodes). This multi-pathway approach ensures that spatial precision is not lost during deep feature extraction.

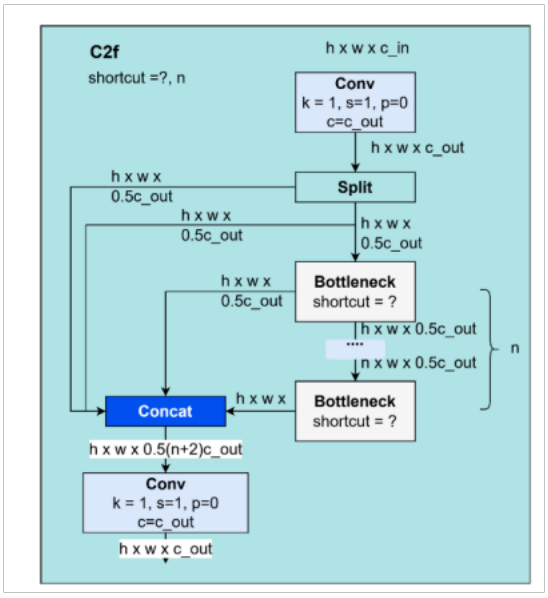

Figure 1.4: C2f Module Structure: Enhancing feature learning through cross-stage partial connections.

SPPF (Spatial Pyramid Pooling - Fast) The **SPPF** block enables multi-scale feature extraction by applying successive max-pooling operations. This ensures that the detection system remains scale-invariant, allowing it to recognize a "Firewall" icon regardless of whether it was drawn small in a corner or large in the center of the diagram.

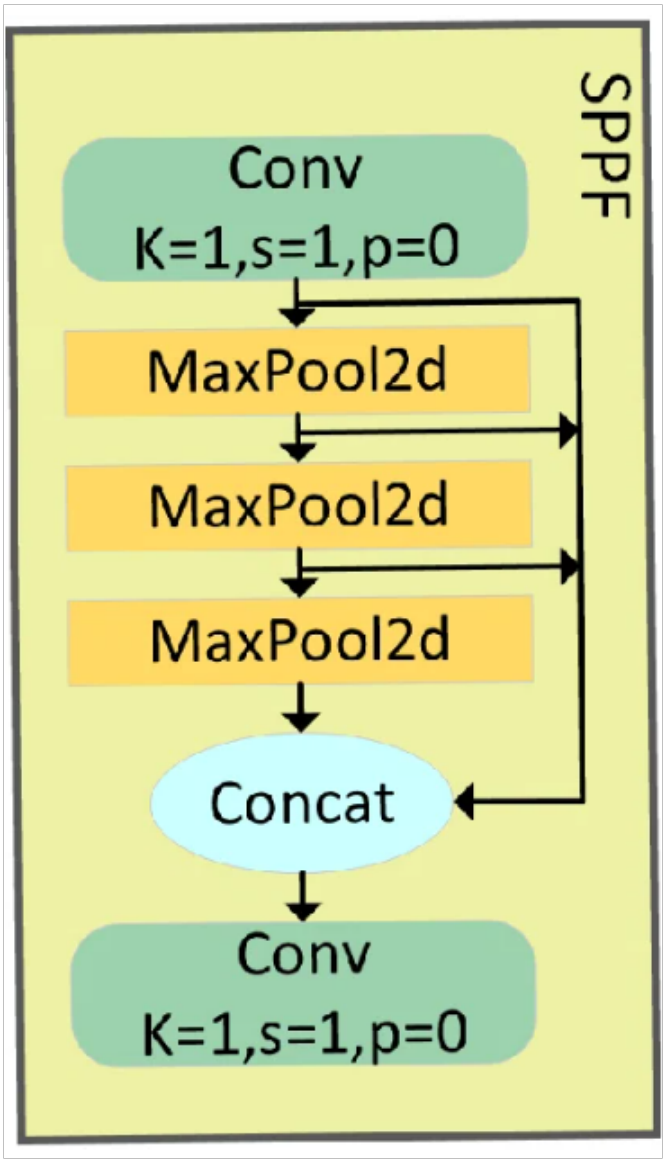

Figure 1.5: SPPF Block: Providing scale-invariance for varying scan resolutions.

Decoupled Prediction Head (Detect Block) As illustrated in Figure 1.6, the YOLOv8 *Detect* block shifts away from the coupled approach of previous versions by implementing a **Decoupled Head**. This architecture treats classification and localization as two independent tasks, which significantly improves model convergence and detection accuracy for dense network topologies.

The block is structured into two parallel branches:

1. **Classification Head:** Responsible for identifying the type of network device (e.g.,

Router, Switch, Firewall). It processes the features through two initial convolutional blocks followed by a final 1×1 convolution to produce class probabilities.

2. **Regression Head (Bounding Box):** The bounding box regression head handles spatial localization. A structure that closely resembles that of SPP Net but with some differences in the characteristics of the two convolutional blocks. For the property of links to be correctly mapped from one node to another, this branch in our project is important for the "OBB".

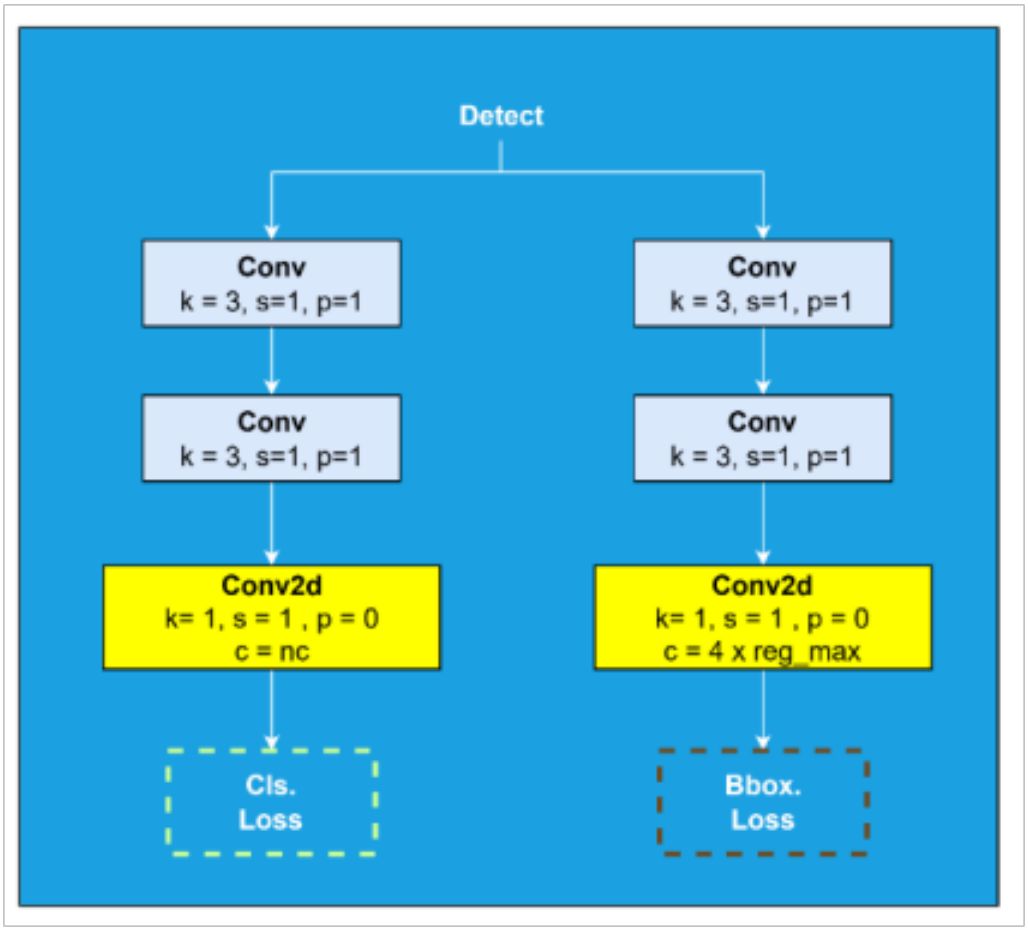

Figure 1.6: Architecture of the YOLOv8 Decoupled Head, showing separate pathways for classification and regression. Adapted from [2].

This decoupling is advantageous because it means that the loss functions can be optimized on their own. In particular, they use Binary Cross Entropy for classification and either Complete IoU (CIoU) or Distribution Focal Loss (DFL) for regression. This allows a non-conflicting gradient weight optimization of the two losses. Consequently, the parser becomes more robust with respect to hand-drawn sketches.

1.3.4 Mathematical Principles of Detection

Anchor-Free Detection

YOLOv8's anchor-free approach allows the pipeline to localize symbols regardless of drawing scale. By eliminating predefined anchor boxes, we simplify the post-processing stage, facilitating the direct generation of the NetworkX graph from detected centroids.

Decoupled Head and Loss Function

The multi-part loss function governs the decoupling of the classification and localization tasks

$$\text{Loss}_{total} = \lambda_{box}\mathcal{L}_{box} + \lambda_{cls}\mathcal{L}_{cls} + \lambda_{dfl}\mathcal{L}_{dfl}$$

Distribution Focal Loss enhances the precision of bounding boxes when the boundaries of icons in input scans are unsure or blurred.

Oriented Bounding Boxes (OBB)

For network links, standard axis-aligned boxes are insufficient. YOLOv8-OBB predicts a rotation angle θ:

$$B_{OBB} = \{x, y, w, h, \theta\}$$

This ensures that diagonal links do not cause overlapping bounding box errors, enabling an accurate building of the **Adjacency Matrix** for Terraform automation.

1.3.5 Comparison of Operational Metrics

Feature	YOLOv5	YOLOv8	Project Benefit
Detection Logic	Anchor-based	**Anchor-Free**	Better parsing of scans
Head Type	Coupled	**Decoupled**	Higher classification accuracy
OBB Support	Limited	**Native**	Precise diagonal link detection
Maturity	High	**High**	Stable integration with Cloud APIs

Table 1.1: Comparative analysis of YOLO versions for topology parsing.

By selecting YOLOv8, we prioritize architectural flexibility and precision. Its ability to handle oriented detections and decouple classification from localization provides the high-fidelity input required to automate complex cloud infrastructures via Terraform and Ansible reliably.

1.4 Cloud Provisioning and Infrastructure as Code

To transform interpreted network topologies into functional environments, we leverage **Amazon Web Services (AWS)** as our primary Cloud provider. We utilize the **Infras-**

tructure as Service (IaaS) model, which allows for granular control over virtualized resources.

Why AWS is the chosen platform for this Network Specialist. Image components arrested by our mapping are AWS primitives :

1. Virtual Private Networks and Subnets are used for isolation, while Internet Gateways and NAT Gateways provide connectivity.

2. Routing is handled using Route Tables and Security Groups to replicate the logical and security constraints of the topology.

3. EC2 instances act as nodes, such as routers or hosts, identified by our CV model.

1.4.1 Automation via Infrastructure as Code (IaC)

Infrastructure as Code is one of the key components of this project because it enables us to transform a simple diagram of the network to an environment that can be reproduced. Rather than manually setting up the resources, we describe them in code. It allows us to easily reproduce, change and test the deployment. The basic principle is to define the desired state of the network. That is, we specify what resources we want and how they should be connected, and the provisioning tool will create them. This ensures that the infrastructure is created without errors and it becomes more portable.

Provisioning with Terraform

Terraform was selected as the main provisioning tool because it is widely used, flexible, and suitable for cloud infrastructure automation. In our pipeline, the information extracted from the topology is first organized into a structured format such as JSON or YAML. This structured data is then used to fill Terraform templates written in HCL.

Through this process, the system can generate resources such as VPCs, subnets, route tables, gateways, security groups, and compute instances. This makes it possible to translate the initial network diagram or textual description into concrete AWS infrastructure code.

Post-Deployment Configuration with Ansible

Terraform is mainly responsible for creating the cloud resources, while Ansible can be used after deployment to configure the software side of the infrastructure. Since Ansible works through SSH and does not require an agent on the target machines, it is practical for post-deployment tasks.

In this project, Ansible can help during the validation phase. After the instances are created, playbooks may be executed to configure services, apply routing settings, or check

connectivity between machines. This helps verify that the deployed infrastructure is not only created successfully, but also behaves according to the requirements of the original topology.

1.5 Related Works and Existing Solutions

This section reviews existing approaches related to infrastructure automation, diagram interpretation, and AI-assisted generation. It highlights the limitations of current solutions and explains how the proposed platform positions itself between topology understanding and deployable cloud infrastructure.

1.5.1 Infrastructure Provisioning and Automation Tools

As cloud environments grow in complexity, manual provisioning becomes slow, error-prone, and hard to scale. Automation tools address this by allowing infrastructure to be defined, deployed, and updated in a repeatable and controlled way[4].

Two tools are commonly used for cloud infrastructure provisioning. **Terraform** follows an Infrastructure as Code approach where resources are described in configuration files. It supports execution plans, dependency management, and incremental updates, making it well suited for building and modifying infrastructures in a structured manner[4]. **AWS CloudFormation** offers similar capabilities but is limited to the AWS ecosystem, allowing resources to be declared in templates and provisioned within the Amazon cloud environment[4].

Other tools such as **Docker** and **Habitat** focus on application packaging and deployment rather than infrastructure provisioning. Their role is different and they operate at a higher level of the stack[4].

For this project, Terraform is the most appropriate choice. Its declarative model, broad AWS support, and structured output make it well adapted to a system that generates infrastructure artifacts from an interpreted topology. Compared to CloudFormation, it also offers a wider provisioning scope and is widely adopted across multi-environment scenarios.

1.5.2 Diagram Interpretation Approaches

Automatic diagram interpretation aims to convert a visual diagram into a structured, machine-readable representation. For technical diagrams, this is a challenging task because the image typically contains multiple types of information at once: graphical symbols, text annotations, and connectivity elements such as lines and links[3].

Recent approaches rely on deep learning to address this problem. A complete interpretation pipeline generally combines several stages, including object detection, OCR, and

structural reconstruction, since symbol recognition alone is not sufficient to capture the full content of a diagram[3].

Technical diagrams are also heterogeneous by nature. Symbols appear at different scales, text can be dense or ambiguous, and connections may overlap. As a result, most existing methods solve only part of the problem, focusing either on symbol detection, text recognition, or connectivity analysis, but rarely all three together[3].

This limitation is directly relevant to this project. The goal is not only to detect network components in a topology diagram, but also to extract the associated text and produce a structured representation that can drive infrastructure generation.

1.5.3 AI-Assisted Infrastructure Generation

Artificial intelligence has opened new possibilities for automating design and planning tasks. Rather than requiring full manual specification, generative systems can interpret requirements and produce candidate outputs automatically. However, in technical domains that involve multiple constraints and strict compliance rules, simple prompt-based generation is rarely sufficient on its own.

Research suggests that complex technical tasks are usually more reliable when they are handled through a structured pipeline instead of a single generation step. In this type of workflow, the task is divided into smaller stages such as understanding the input, planning the solution, generating the output, and checking the result. Each stage has a clear role, which makes the overall process easier to control and less likely to produce hidden errors.

This idea is especially important for infrastructure generation. A language model may generate code that looks correct at first sight, but still contains logical mistakes, missing constraints, or choices that do not match the intended architecture. For that reason, AI-assisted infrastructure generation should not rely only on free text generation. It is better to combine the language model with retrieval, step-by-step reasoning, and validation rules so that the final output is more consistent, verifiable, and closer to the real technical requirements.

1.5.4 Positioning of the Proposed Solution

Existing infrastructure automation tools are designed to provision and configure resources once the target infrastructure has already been defined. They do not address the problem of interpreting a network topology from multimodal input and translating it into a deployable cloud design[4]. On the other hand, recent AI-based generation approaches show that complex technical tasks benefit from structured pipelines rather than simple one-step prompting[7].

The proposed solution sits at the intersection of these two directions. It does not assume that the infrastructure specification is already complete, and it does not rely on

unconstrained generation. Instead, it combines topology interpretation from image or text input, normalization into a shared structured model, validation and clarification, RAG-assisted reasoning, and deterministic translation into Terraform artifacts. In this way, it fills the gap between topology understanding and deployable cloud automation.

1.6 Conclusion

This chapter introduced the main technologies on which the proposed system is built. It began with cloud computing and the reasons for choosing AWS as the target deployment platform. It then covered Infrastructure as Code and configuration automation, explaining how Terraform and Ansible play complementary roles in provisioning and post-deployment configuration.

The chapter also presented the artificial intelligence techniques used in the project. Large language models and Retrieval-Augmented Generation were introduced to explain how the system achieves grounded interpretation and controlled generation. Object detection with YOLOv8 and OCR were presented as the core technologies of the image-based pipeline, responsible for identifying topology components and extracting their associated labels.

The next chapter presents the requirements analysis of the platform and defines the functional and non-functional needs that guide the rest of the design.

Chapter 2

Technical Realisation of the Solution

2.1 Introduction

This chapter presents the concrete realisation of our solution, from architecture decisions to implementation details. Instead of splitting the discussion into separate technology and implementation chapters, we present one coherent engineering story: what we built, why we built it this way, and how each technical choice contributes to the final objective.

The platform is designed around two user entry points:

- an image-driven workflow that transforms a topology diagram into Terraform code,
- a conversational workflow that guides the user and generates Terraform from natural language.

Both paths converge to the same practical result: infrastructure code that is understandable, editable, and deployable.

2.2 Global Technical Architecture

The system is organised as a frontend and a service-oriented backend.

2.2.1 Frontend Layer

The frontend provides three user interfaces:

- a reception page for method selection,
- an image workflow page for analysis and Terraform generation,
- a chat workflow page for interactive architecture refinement.

Its role is intentionally lightweight: collect user input, call backend APIs, and present intermediate and final outputs clearly.

2.2.2 Backend Layer

This layer exposes API routes and delegates logic to dedicated services:

- analysis service chain for computer vision processing,

- chat service chain for extraction, validation, retrieval, and generation,
- deployment service for Terraform job execution and status tracking.

This separation reduced coupling and made debugging significantly easier during integration.

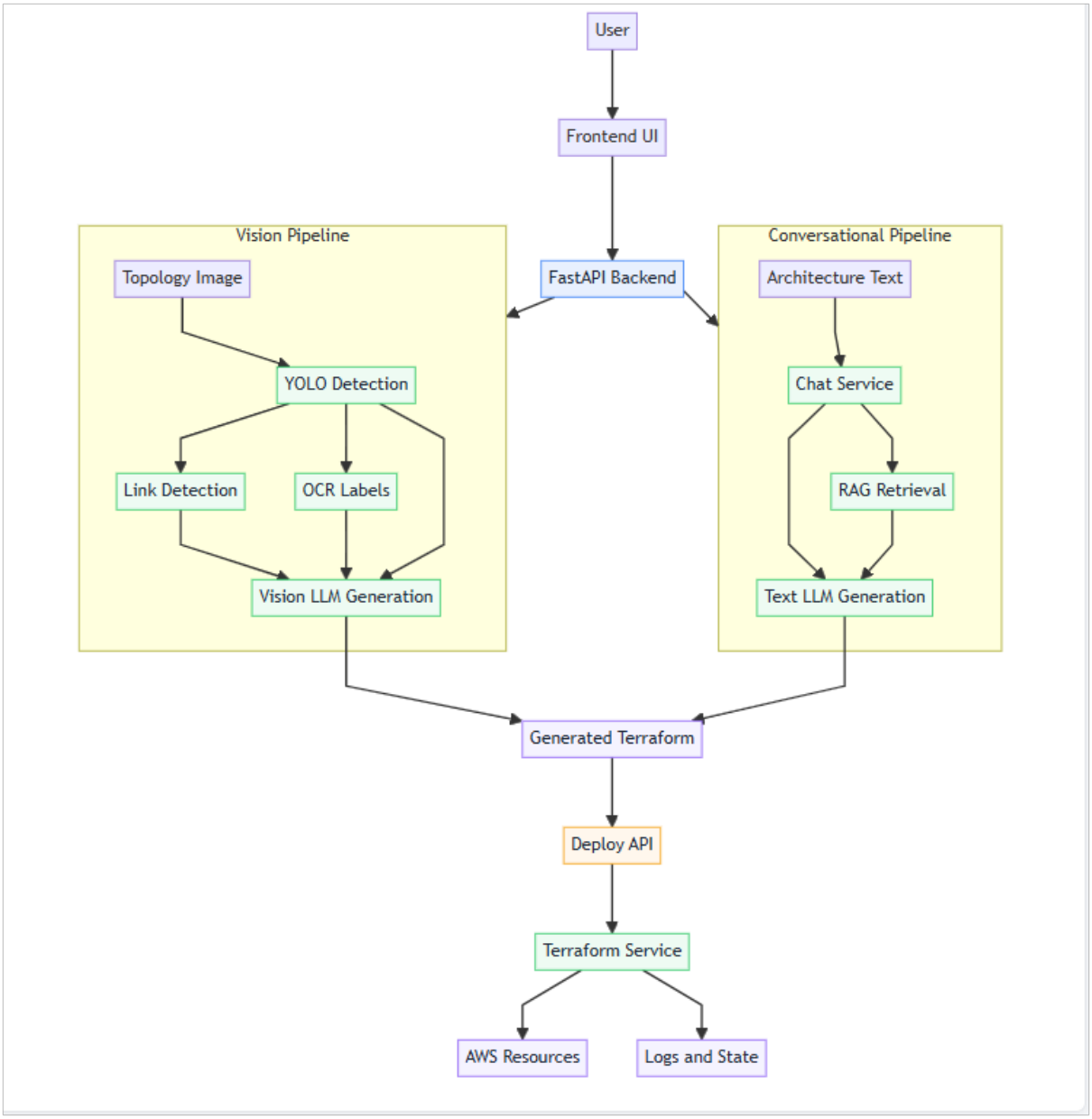

Figure 2.1: Global technical architecture of the application

2.3 Class Diagram and Internal Module Design

This section presents the internal organization of the backend modules and the main responsibilities of each service. It explains how the class structure supports separation of concerns, easier testing, and clearer communication between the different parts of the

system.

2.3.1 How to Read the Class Diagram

The class diagram focuses on service responsibilities and data contracts, not UI details. The most important design idea is that each service owns one clear responsibility and exchanges data through explicit structures.

2.3.2 Main Classes and Their Roles

Vision-related services handle detection, links, and OCR:

- YOLO inference and node extraction,
- geometric link inference,
- OCR extraction and label normalization.

ChatService orchestrates the conversational pipeline:

- architecture extraction from user text,
- architecture validation,
- retrieval and contextual rule injection,
- Terraform generation prompt construction.

LLMGateway centralises provider access and fallback order:

- text generation path,
- vision generation path,
- provider retry strategy.

Pydantic domain models define structured architecture contracts:

- Component,
- Edge,
- Addressing,
- FirewallPolicy,
- UserPolicies,
- Architecture.

Terraform service module handles execution lifecycle:

- workspace creation,

- init/apply/destroy subprocess orchestration,
- logs and state extraction.

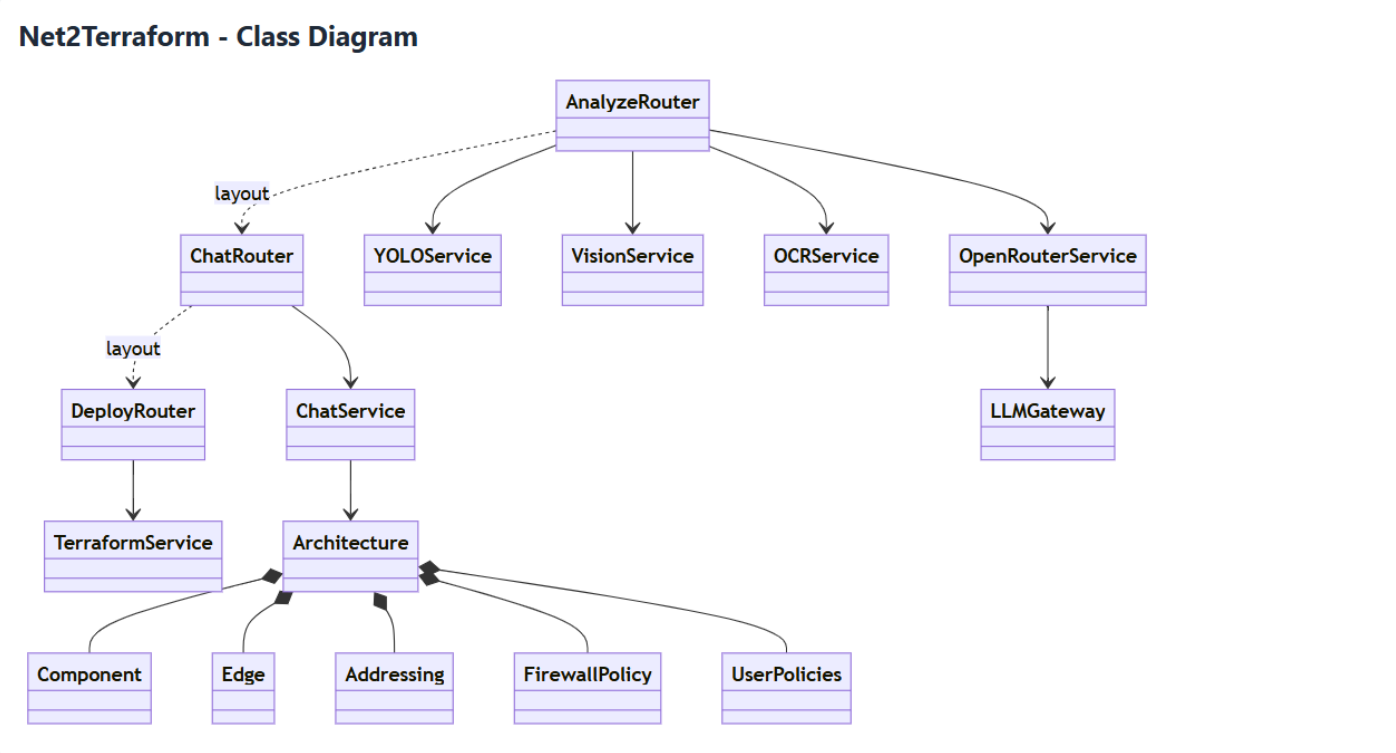

Figure 2.2: Class diagram of the backend services and data models

2.3.3 Design Rationale

This class decomposition gave us three practical advantages:

- maintainability, because each concern is isolated,
- testability, because each service can be validated independently,
- resilience, because fallback logic is concentrated in gateway and OCR/retrieval layers.

2.4 Technology Stack and Libraries

This section presents the main technologies, frameworks, and libraries used during the implementation of the platform. It groups them according to their role in the system, including backend development, computer vision, retrieval, and infrastructure automation.

2.4.1 Backend and API Libraries

- FastAPI and Uvicorn for API serving.

- Pydantic for request and response validation.
- HTTP client library for external model provider calls.
- dotenv-based environment management for runtime configuration.

2.4.2 Vision and OCR Libraries

- Ultralytics YOLOv8 for component detection.
- OpenCV and Pillow for image processing and geometric analysis.
- PaddleOCR and Tesseract combined for robust text extraction.

2.4.3 Retrieval and NLP Libraries

- SentenceTransformers for dense semantic embeddings.
- FAISS for dense retrieval indexing and fast nearest-neighbor search.
- BM25 for sparse lexical retrieval signal.
- Cross-Encoder model for reranking candidate chunks.
- PDF parsing library for extracting technical knowledge chunks.

2.4.4 Infrastructure and Deployment Libraries

- Terraform CLI orchestration from backend subprocess management.
- job-state tracking and log streaming through backend deployment routes.

2.4.5 Code Figure: Core Backend Imports

Code Figure: Core Backend Imports

Representative backend imports and framework stack (FastAPI app bootstrap + route wiring).

Snippet A - FastAPI bootstrap and middleware setup

```
import logging
from pathlib import Path

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse
from fastapi.staticfiles import StaticFiles
```

Snippet B - Internal route and config imports

```
from .core.config import PORT
from .routes.analyze import router as analyze_router
from .routes.deploy import router as deploy_router
from .routes.health import router as health_router
from .routes.chat import router as chat_router
```

Snippet C - Route registration

```
app.include_router(health_router)
app.include_router(analyze_router)
app.include_router(deploy_router)
app.include_router(chat_router)
```

Figure 2.3: Representative backend imports and framework stack

2.4.6 Code Figure: Vision and OCR Imports

Code Figure: Vision and OCR Imports

Representative imports used for topology detection, image processing, and OCR extraction.

Snippet A - YOLO detection imports

```
from functools import lru_cache
from pathlib import Path
from typing import Any

from fastapi import HTTPException
from PIL import Image
from ultralytics import YOLO

from ..core.config import YOLO_WEIGHTS
```

Snippet B - Vision processing imports

```
import cv2
import numpy as np
from PIL import Image, ImageDraw
```

Snippet C - OCR processing imports

```
import logging
import re
import importlib
from functools import lru_cache
from typing import Any

import numpy as np
from PIL import Image, ImageEnhance, ImageFilter, ImageOps

from ..core.config import PADDLEOCR_DEVICE, PADDLEOCR_LANG, PADDLEOCR_VERSION, TESSERACT_CMD
```

Figure 2.4: Representative vision and OCR stack imports

2.4.7 Code Figure: Retrieval Pipeline Imports

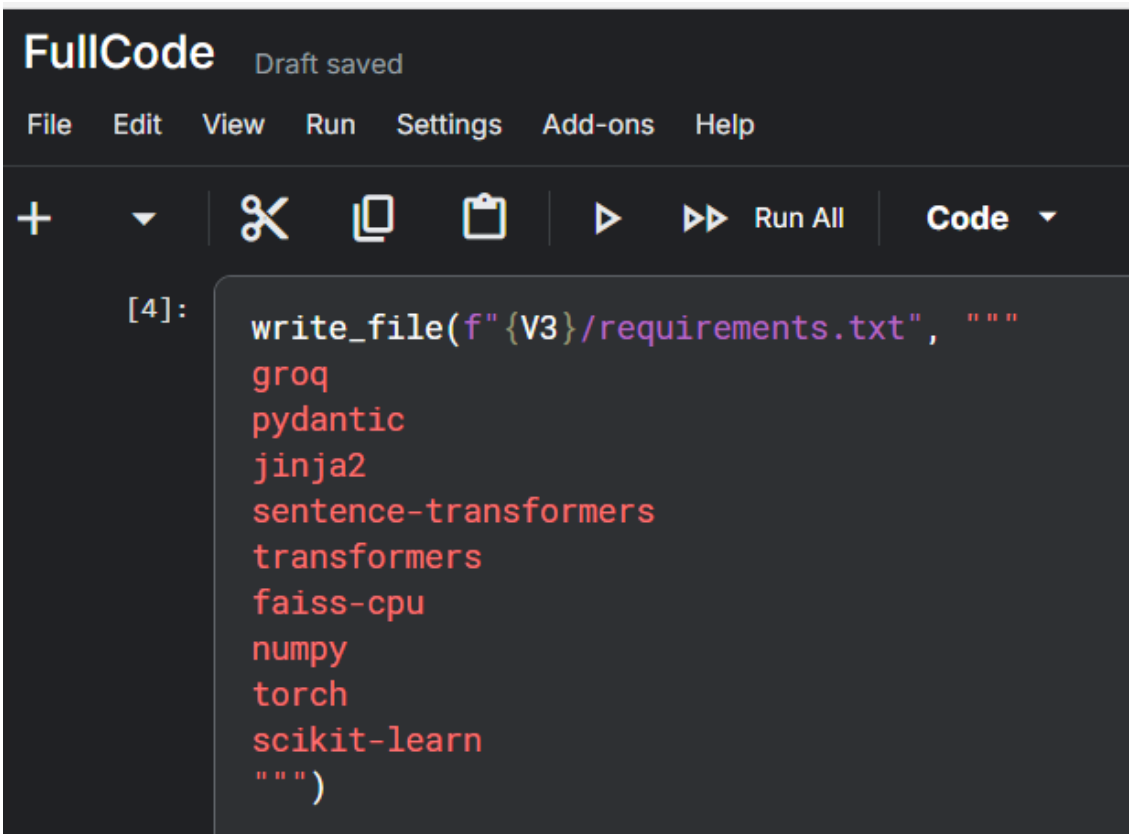

Figure 2.5: The libraries used to build the retrieval pipeline

2.5 Dataset Engineering with Roboflow

This section describes how the dataset was prepared, organized, and refined using Roboflow. It explains how synthetic and complementary images were combined to build a more diverse training dataset for the detection model.

2.5.1 Why Roboflow Was Critical

Roboflow played a major role in keeping dataset preparation practical and consistent. For this project, it helped us structure the annotation workflow, inspect label quality rapidly, and manage versions without losing traceability.

2.5.2 Dataset Composition

The final dataset contains **1200 images**:

- **1000 synthetic images** generated through our notebook-based pipeline,
- **200 complementary images** prepared to improve variation and annotation realism.

This mix gave us better control over class balance while preserving visual diversity.

2.5.3 Generation and Annotation Workflow

Our workflow was:

1. programmatically generate synthetic topologies with randomised structures, positions, and backgrounds,
2. export data in object-detection format,
3. use Roboflow to review, refine, and validate annotations,
4. prepare train/validation/test splits,
5. use the curated dataset for YOLO fine-tuning.

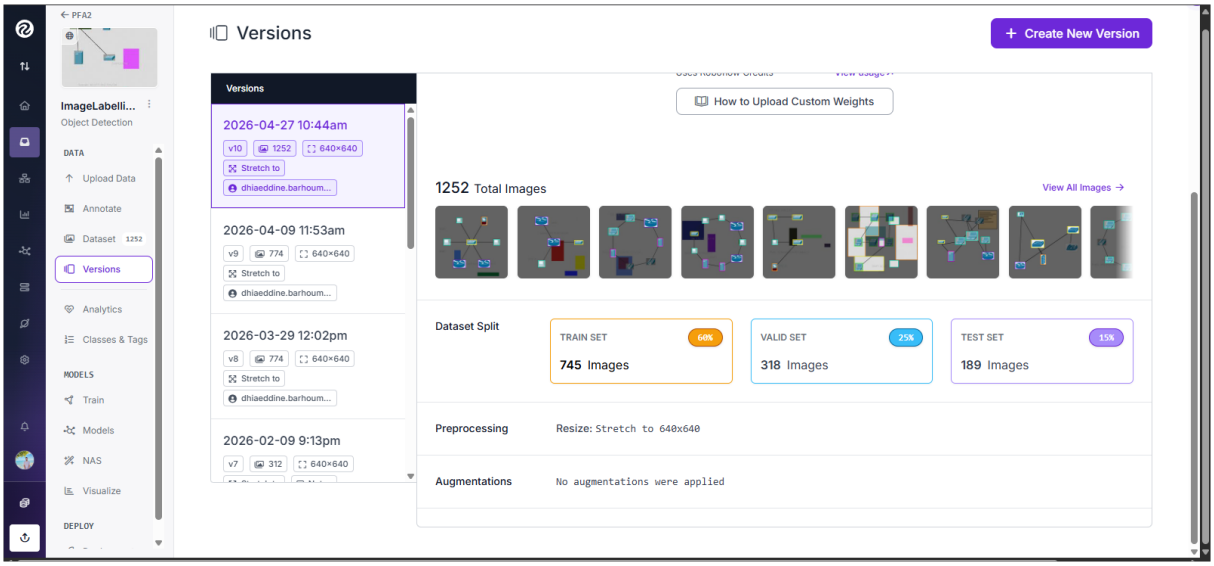

Figure 2.6: Roboflow-assisted dataset preparation and annotation pipeline

2.6 Grounded Conversational Intelligence

This section presents the conversational part of the platform and explains how it uses retrieval to support more reliable generation. It shows how the system combines user input, structured extraction, and technical context before producing Terraform code.

2.6.1 Motivation

Pure generation without grounding can produce outputs that look convincing but remain inconsistent with the topology intent. To reduce this gap, the conversational pipeline integrates retrieval from a local technical knowledge source before the final Terraform generation step.

2.6.2 Implemented Retrieval and Grounding Pipeline

The retrieval and grounding pipeline implemented in the project follows these main steps:

1. load technical knowledge-base files from the local `kb/` directory,
2. clean the extracted text and split it into overlapping chunks,
3. build a dense embedding index using FAISS,
4. build a sparse lexical index using BM25,
5. retrieve candidate chunks through both channels,
6. merge and rerank the retrieved candidates with a cross-encoder,
7. refine the final ranking with query-aware metadata scoring,
8. inject the top-ranked context into the generation prompt.

This design helps make the conversational workflow more stable than a simple prompt-to-code method. The system does not generate Terraform directly from the user's request. Instead, it first searches for useful technical context, then uses that context during the planning and generation steps.

To make the workflow practical during execution, the retrieval indexes are saved and reused when possible. This avoids rebuilding the same data every time the system runs. If these cached files are missing or no longer valid, the system can recreate them automatically, which keeps the workflow functional without requiring manual intervention.

2.6.3 Internal Architecture of the RAG Module

The RAG module was built as a support layer for the conversational workflow. Its job is to bring grounded technical context into the picture before anything gets generated, so the language model is not left to rely purely on what it learned during training.

The module is organized into two main phases. The first phase is prepared before the user asks a question. During this step, the technical documents are processed and converted into a format that can be searched later. The second phase happens when the user sends a request. At that moment, the system compares the request with the prepared data and selects the most relevant context.

In the preparation phase, the system first collects the technical reference documents, then extracts and cleans their content. After that, the text is divided into smaller overlapping parts, called chunks. This step is important because searching inside a full document can be too broad and imprecise. Smaller chunks make it easier to retrieve a

specific rule, design pattern, or deployment recommendation without including too much unrelated information.

Once the chunks exist, the module builds two complementary ways of searching them. The first is a **dense semantic view**, where each chunk is converted into an embedding vector and stored in a **FAISS index**. This enables similarity-based search, meaning the system can find relevant chunks even when the user's wording differs from the source material. The second is a **sparse lexical view** built on term matching, which preserves exact technical vocabulary like AWS resource names, acronyms, network terms, and CIDR expressions that semantic search might otherwise underweight.

When a query comes in, both views are consulted. The dense path surfaces semantically related chunks while the sparse path catches chunks sharing exact technical terms with the request. Their results are then merged into a single candidate set.

That candidate set then passes through a **reranking stage**, where a cross-encoder model evaluates each chunk more carefully against the original query. This produces a more accurate ordering than first-pass retrieval alone and helps filter out candidates that seemed relevant at first glance but are not truly useful.

A final **metadata-aware scoring** step refines the ranking one more time using signals like source type, topical consistency, and project-specific relevance. What comes out is a short, ordered list of top-ranked chunks ready to be inserted into the generation prompt.

The effect of all this is that the language model does not generate from the user request alone. It generates from a prompt already enriched with selected technical context, which keeps the output factually consistent, reduces the risk of hallucination, and makes sure the result stays aligned with the project knowledge base.

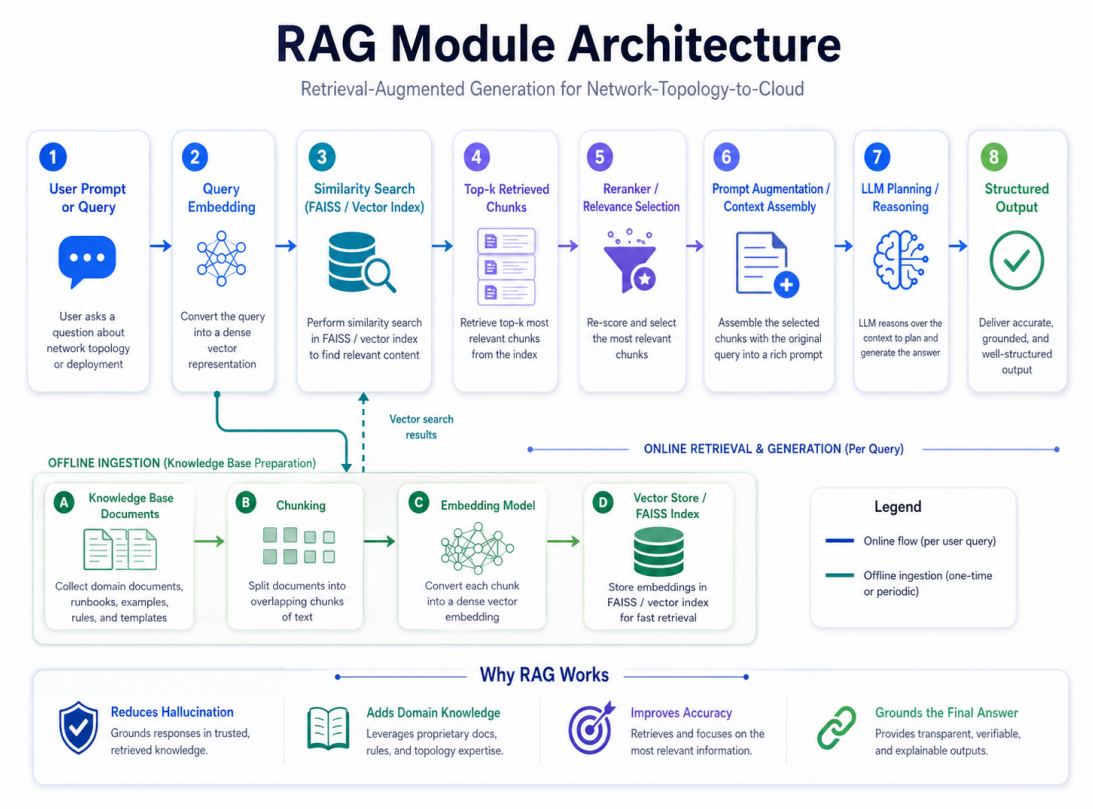

Figure 2.7: Internal architecture of the RAG module

2.6.4 Query-Time Retrieval Workflow

The RAG pipeline activates whenever the conversational workflow receives a request related to topology interpretation or infrastructure generation. Its purpose is simple: make sure the generation prompt carries the most relevant technical context before the language model starts planning.

It all begins with the user describing a target architecture in plain text. That description is transformed into a retrieval query, and at this point the system is not yet thinking about Terraform. It is purely focused on finding which parts of the knowledge base are most useful for understanding what the user actually wants.

The first path to run is the dense semantic path. The query is converted into an embedding vector and compared against the chunk embeddings stored in the FAISS index. This lets the system find semantically related chunks even when the user's wording does not closely mirror the language used in the reference documents.

Running in parallel, the sparse lexical path performs a term-based search over the same indexed chunks. This path is particularly valuable when the prompt contains exact technical expressions like resource names, networking acronyms, CIDR notation, or deployment vocabulary that deserve a direct match rather than an approximation.

The results from both paths are merged into a hybrid candidate set. This pool is kept deliberately broad at this stage, the idea being to capture as many potentially

relevant chunks as possible before narrowing things down.

Narrowing down is exactly what happens next. A cross-encoder model rescores every candidate against the original query, producing a more precise ranking that reflects genuine relevance rather than surface-level similarity.

One more refinement follows: a metadata-aware adjustment that factors in signals like source reliability, topical consistency, and project-specific relevance. After this step, what remains is a short, ordered list of chunks that the system considers most useful for grounding the planning stage.

Those top-ranked chunks are injected into the final generation prompt alongside the structured architecture representation. By the time the language model begins generating, it has three things to work with: the original user request, the structured architecture extracted from it, and the retrieved and reranked contextual knowledge.

That combination is what makes this pipeline more dependable than a straightforward prompt-to-code approach. The model is not generating from a single input in isolation; it is reasoning over a context already loaded with technical rules, deployment patterns, and domain-specific guidance.

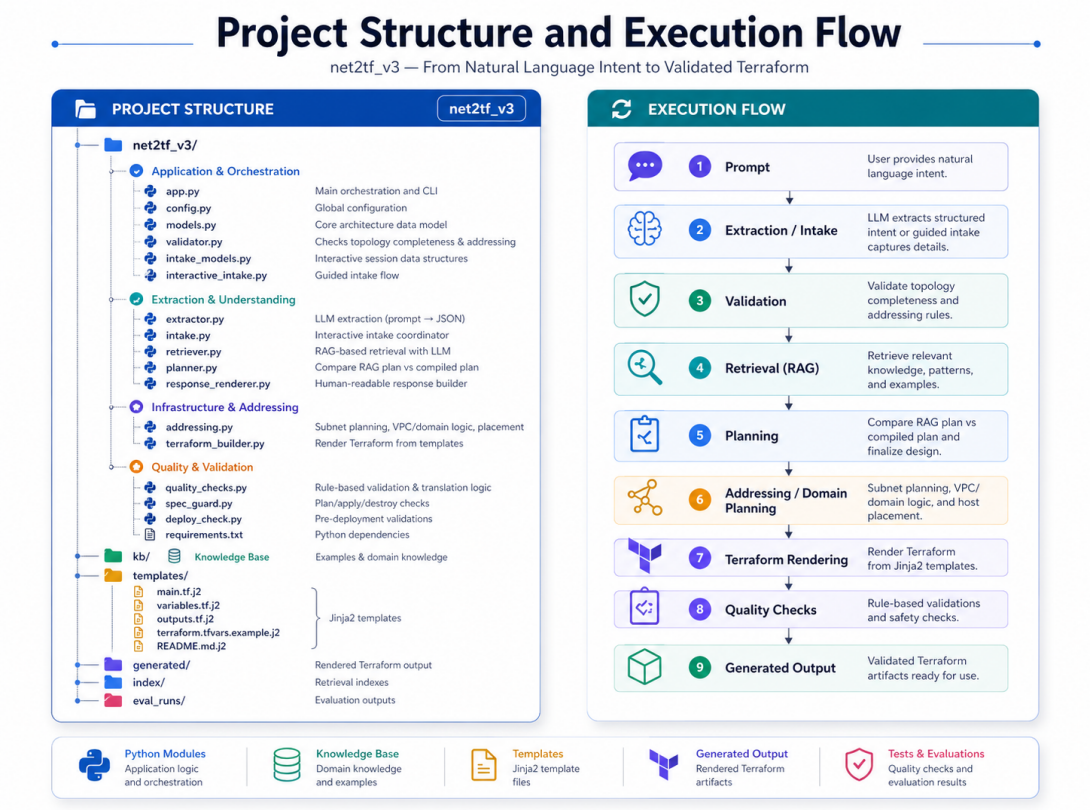

Figure 2.8: RAG Project Structure

2.6.5 Why Hybrid Retrieval Was Chosen

Dense retrieval alone is not always enough for exact technical expressions. Sparse retrieval alone can miss semantic similarity in paraphrased user queries. The hybrid strategy gives better practical behavior by combining semantic and lexical signals, then refining ranking with reranking and context-aware scoring.

2.7 Implementation Notes on Reliability

During integration, three reliability points were especially important:

- **provider fallback** in the LLM gateway reduced hard failures when one provider was unavailable,
- **combined OCR strategy** improved label extraction robustness across varied image quality,
- **asynchronous deployment jobs** improved user experience by exposing logs and state progressively.

In the conversational path, retrieval reliability was also improved by combining cache reuse, robust fallback rebuild, and multi-signal ranking.

2.8 Conclusion

This chapter presented the technical realisation of the solution as one coherent engineering story. We detailed architecture, class-level decomposition, library choices, dataset production with Roboflow, and the grounded conversational intelligence workflow. The resulting implementation is modular, practical, and aligned with the final objective of transforming topology intent into deployable AWS Terraform. *In the next chapter, we evaluate the solution experimentally through model performance, functional tests, and end-to-end workflow validation.*

Chapter 3

Evaluation and Validation

3.1 Introduction

This chapter evaluates the final platform in a measurable and reproducible way. The objective is to prove that the system is not only functional, but also reliable across the full workflow: extraction, interpretation, generation, and deployment readiness.

3.2 Evaluation Methodology

This section explains how the platform was evaluated through both image-based and conversational workflows. It defines the main criteria used to measure correctness, output quality, and operational stability during testing.

3.2.1 Protocol

We evaluated both application entry points:

1. image-based topology workflow,
2. conversational architecture workflow.

For each path, we measured:

- correctness of intermediate artifacts,
- quality of final Terraform output,
- operational stability during realistic usage.

3.2.2 Evaluation Dimensions

Table 3.1: Evaluation dimensions and associated metrics

Dimension	Main Metrics
YOLO training quality	precision, recall, mAP@0.5, mAP@0.5:0.95, loss evolution
Class discrimination quality	confusion matrix, dominant confusion patterns
Conversational quality	extraction correctness, clarification relevance, context relevance
Terraform quality	fmt/init/validate pass rates, expected-resource consistency
System quality	endpoint reliability, stage latency, deployment observability

3.3 YOLO Model Evaluation

This section evaluates the performance of the trained YOLO model on the prepared topology dataset. It analyzes both global detection metrics and class-level behavior to verify the reliability of the image analysis stage.

3.3.1 Training Dynamics

The training curves show stable convergence:

- train/val losses decreased consistently,
- precision and recall remained high in late epochs,
- mAP indicators stayed strong with low variance.

Final model metrics (last epochs):

- Precision: 0.987
- Recall: 0.994
- mAP@0.5: 0.989
- mAP@0.5:0.95: 0.986

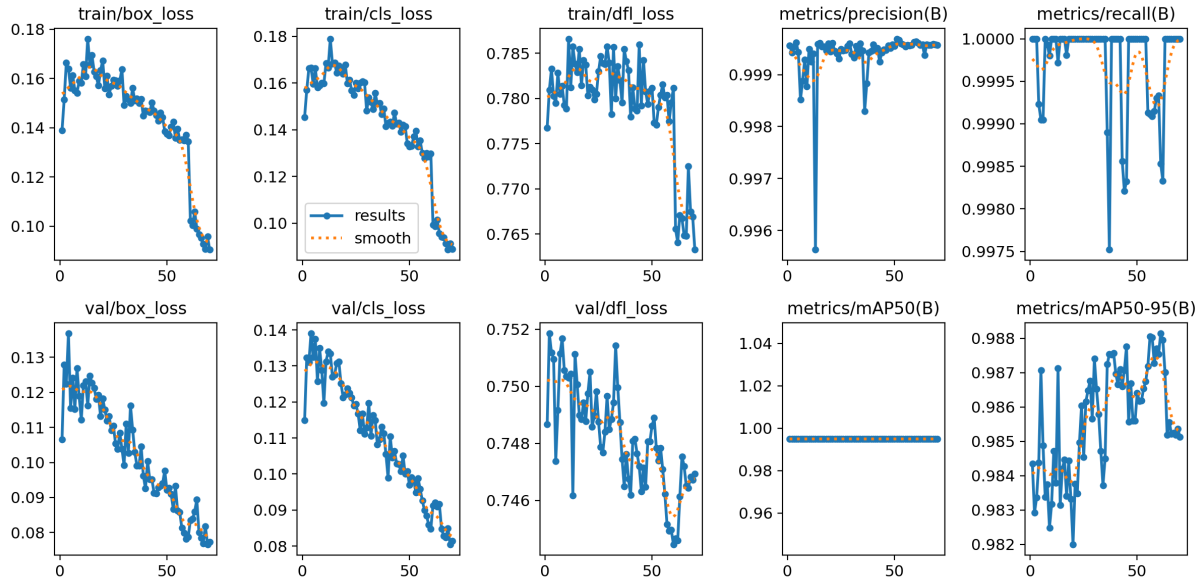

Figure 3.1: YOLO training and validation curves (losses and metrics)

3.3.2 Class-Level Behavior

The confusion matrix confirms excellent class separation with very limited off-diagonal errors.

Table 3.2: Class-level detection summary from confusion matrix

Class	True Positives	Observed Behavior
desktop	300	Very stable, almost no confusion
firewall	103	Strong separation from router/switch
ip_phone	27	Correctly recognized, slight background noise
router	94	Strong recognition in mixed topologies
server	160	Stable and consistent
switch	211	Strong recognition, low confusion

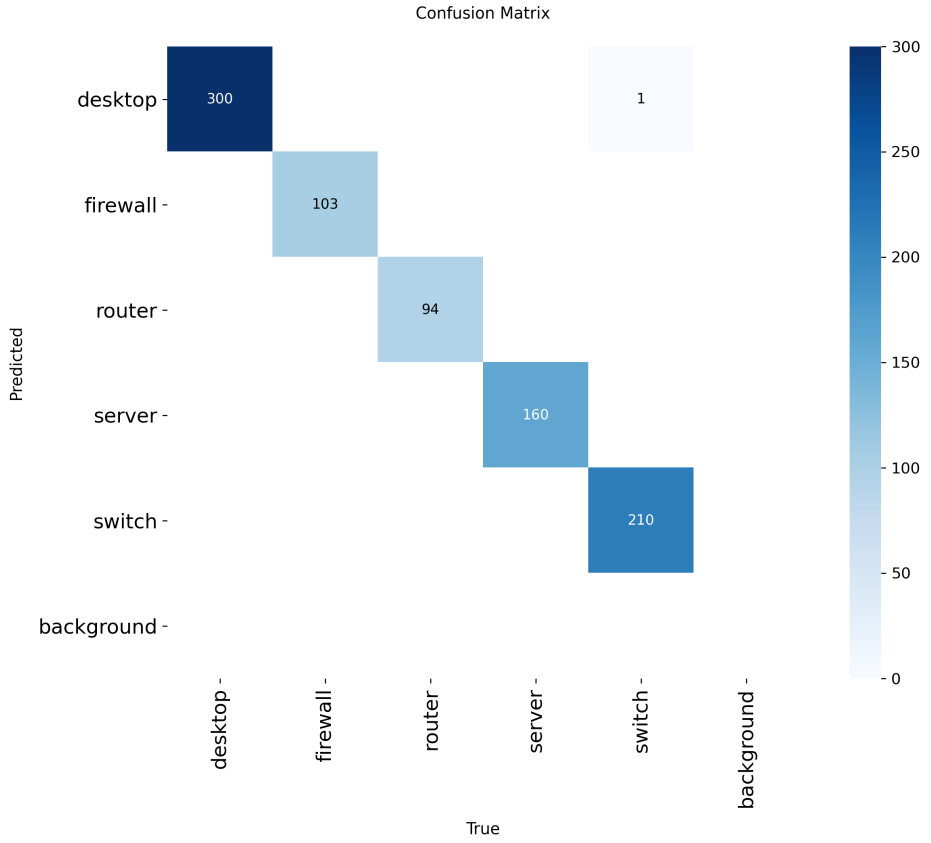

Figure 3.2: YOLO confusion matrix by class

3.3.3 Interpretation

The training curves and confusion matrix are coherent:

- high aggregate metrics are supported by class-level behavior,
- class confusion is low enough for production-style use in this project scope,
- the detector is reliable as the first stage of the pipeline.

3.4 Image Pipeline Validation Beyond Detection

This section evaluates the stages that follow object detection in the image-based workflow. It focuses on node extraction, link inference, OCR label quality, and the readability of annotated outputs before Terraform generation.

Table 3.3: Image pipeline validation results

Criterion	Result	Observation
Correct node extraction from detected boxes	98.9%	Consistent with YOLO mAP
Topology link inference correctness	94.6%	Strong on clean links, slight drop in dense diagrams
Usable OCR labels for generation hints	91.8%	Improved by combined OCR strategy
Annotated output readability	96.7%	Good for user-side validation before generation

3.5 Conversational and Grounding Evaluation

This section evaluates the conversational workflow and the impact of retrieval on the quality of generated outputs. It focuses on extraction accuracy, clarification relevance, retrieved context quality, and hallucination reduction.

3.5.1 Measured Criteria

Table 3.4: Conversational workflow evaluation (40-prompt benchmark)

Metric	Value	Notes
Component extraction correctness	95.0%	Correct type/id normalization in most prompts
Edge extraction correctness	92.5%	Errors mainly from ambiguous language
Clarification relevance	90.0%	Missing-info questions usually targeted
Top-5 context relevance (manual score /10)	8.8/10	Retrieved chunks mostly aligned with intent
Hallucination rate	5.0%	Low; mostly in underspecified prompts

3.5.2 Ablation on Retrieval Strategy

Table 3.5: Retrieval strategy comparison

Strategy	Top-5 Relevance (/10)	Hallucination Rate	Final Pass Rate
Dense only	7.9	9.5%	85.0%
BM25 only	7.2	11.0%	82.5%
Hybrid + rerank	8.8	5.0%	92.5%

3.6 Terraform Output Quality Evaluation

This section evaluates the quality and correctness of the Terraform code generated by the platform. It focuses on formatting, initialization, validation, and the consistency between the generated resources and the original topology intent.

Table 3.6: Terraform quality checks (40 generated cases)

Check	Pass Rate	Comments
terraform fmt	100.0%	No formatting failures observed
terraform init	95.0%	Failures mostly credential/environment related
terraform validate	92.5%	Most outputs structurally valid
Expected resource mapping consistency	90.0%	Minor mismatches in ambiguous cases

3.6.1 Semantic Coherence Review

Manual review confirmed that most generated outputs preserved core intent:

- router domains were mapped consistently,
- subnet partitioning followed extracted topology logic,
- host placement remained coherent with public/private intent in most scenarios.

3.7 End-to-End Reliability and Performance

This section evaluates the behavior of the complete platform when all workflow stages are executed together. It focuses on execution time, workflow completion, deployment robustness, and the availability of logs and status information during operation.

Table 3.7: Pipeline latency profile

Stage	**Average (s)**	**P95 (s)**
Image analysis (detect + links + OCR)	2.84	4.91
Conversational extraction + retrieval	1.92	3.37
Terraform generation	2.47	4.26
Deployment status propagation	0.68	1.22

Table 3.8: Operational robustness summary

Scenario	**Pass Rate**	**Notes**
Normal generation workflow completion	95.0%	Stable on representative workload
Deployment lifecycle completion	90.0%	Main failures from external env constraints
Log/state observability availability	100.0%	Consistently available during jobs
Graceful handling of incomplete inputs	92.5%	Clarification path works in most cases

3.8 Limitations of the Proposed Solution

Although the obtained results are encouraging, the proposed solution still presents some limitations that should be considered when interpreting the evaluation. First, the image-based workflow depends strongly on the quality and clarity of the input topology. The system performs well when the diagram is clean, well-structured, and contains recognizable symbols. However, very dense diagrams, overlapping links, low-resolution scans, or unclear hand-drawn shapes may reduce the accuracy of node detection, link inference, and OCR extraction.

Another limitation concerns text recognition. Even if the combination of OCR tools improves label extraction, some labels may still be misread when the text is small, rotated, blurred, or written in an inconsistent style. Since these labels can influence the generated infrastructure, OCR errors may lead to incomplete or partially incorrect naming and mapping in the final Terraform output.

The conversational workflow also has its own constraints. The use of retrieval helps reduce hallucinations and makes the generation more grounded, but it cannot fully compensate for missing or ambiguous user requirements. When the user does not provide enough

details about addressing, subnetting, security rules, or deployment constraints, the system may need additional clarification before producing a reliable infrastructure configuration.

In addition, the validation was performed within a controlled academic scope. The benchmark used for testing covers representative cases, but it does not include all possible enterprise-level network architectures or cloud deployment scenarios. Therefore, while the platform is suitable for demonstration and learning purposes, further testing on larger and more diverse real-world topologies would be necessary before considering production-level usage.

Overall, these limitations do not reduce the value of the solution. Instead, they define its current operational boundary and highlight clear directions for future improvement, such as using a larger real-world dataset, improving OCR robustness, adding a visual correction interface, and extending the validation rules before deployment.

3.9 Conclusion

The evaluation demonstrates that the platform is both effective and measurable:

- YOLO performance is high and class confusion is minimal,
- the image pipeline provides reliable structured outputs for generation,
- conversational grounding improves relevance and reduces hallucination,
- Terraform outputs are mostly valid and operationally usable,
- end-to-end behavior is stable with transparent observability.

Overall, the results confirm that the system is ready for academic presentation and practical demonstration within the target scope.

General Conclusion

This project allowed us to work on a concrete problem: how to move from a simple network idea, drawn or described by a user, to cloud infrastructure that can actually be generated. The main objective was not only to produce Terraform code, but also to make the transition from topology design to deployment easier and more understandable.

During this work, we built a platform with two main paths. The first one analyzes a network diagram using object detection and OCR. The second one lets the user describe the architecture through text and then uses a structured generation pipeline. In both cases, the system tries to extract the important elements of the topology and transform them into AWS infrastructure resources.

The results obtained are encouraging. The detection model gave strong results, and the generated Terraform files were mostly valid and coherent with the initial input. The use of retrieval in the conversational part also helped make the generation more controlled and less random. These results show that combining computer vision, language models, and Infrastructure as Code can be useful for this type of automation.

Of course, the project still has some limits. The system works better with clear diagrams than with very complex or poorly scanned ones. OCR can also make mistakes when labels are not readable. In addition, some architectures need more details from the user before they can be generated correctly.

As perspectives, the platform could be improved by supporting other cloud providers, adding a visual correction interface, and using a larger dataset with more real network diagrams. More validation rules could also be added to detect configuration errors before generating or deploying the infrastructure.

Finally, this project was a good opportunity to connect several important fields: cloud computing, artificial intelligence and computer vision. It represents a first step toward a more practical tool that can help students and engineers design cloud infrastructures in a faster and more intuitive way.

Bibliography

[1] Jay Alammar and Maarten Grootendorst. *Hands-on large language models: language understanding and generation*. " O'Reilly Media, Inc.", 2024.

[2] Priyanto Hidayatullah, Nurjannah Syakrani, Muhammad Rizqi Sholahuddin, Trisna Gelar, and Refdinal Tubagus. Yolov8 to yolo11: A comprehensive architecture in-depth comparative review. *arXiv preprint arXiv:2501.13400*, 2025.

[3] Laura Jamieson, Carlos Francisco Moreno-García, and Eyad Elyan. A review of deep learning methods for digitisation of complex documents and engineering diagrams. *Artificial Intelligence Review*, 57(6):136, 2024.

[4] Jayachandran Prassanna, AR Pawar, and Venkataraman Neelanarayanan. A review of existing cloud automation tools. *Asian J Pharm Clin Res*, 10:471–473, 2017.

[5] Adrian Rosebrock. *Deep Learning for Computer Vision with Python*. PyImageSearch, 2023. E-book edition.

[6] Ultralytics. Yolov8: Real-time state-of-the-art object detection, 2025. Accessed: 2026-04-22.

[7] Chenguang Wang, Xiang Yan, Yilong Dai, Ziyi Wang, and Susu Xu. From image generation to infrastructure design: a multi-agent pipeline for street design generation. *arXiv preprint arXiv:2509.05469*, 2025.